

Kolibri CRM — Manual Operativo de Datos

Entregable 2 · Ecosistema de Automatización IA Autónomo · Entrega Final — Coderhouse
AI Automation

Este manual documenta las tres bases de datos de Notion que sostienen el ecosistema de automatización de Kolibri CRM, su propósito dentro del flujo de n8n, la estructura de campos de cada una y los procedimientos operativos para mantenerlas y auditarlas. Es la referencia para cualquier persona del equipo (o agencia) que necesite operar, dar soporte o extender el sistema sin depender de quien lo construyó.

1. Visión general de las bases de datos

El sistema usa Notion como capa de datos (base de datos operativa) para tres funciones separadas: el contenido en proceso de generación y aprobación, el conocimiento de referencia (RAG) que la IA usa como contexto, y el registro de errores para trazabilidad y resiliencia. Las tres bases viven en el workspace **Kolibri CRM – Ecosistema IA** y se conectan entre sí únicamente a través del flujo de n8n (no hay relaciones nativas de Notion entre ellas).

Base de datos	Propósito	Acceso desde n8n
Centro de Comando	Piezas de contenido generadas por IA, su estado de aprobación y el contenido final.	n8n: escribe, actualiza y lee (trigger)
Base RAG	Conocimiento de referencia de la marca/negocio que se inyecta como contexto en el prompt.	n8n: solo lectura (Get Many)
Log de Errores	Registro de cada falla ocurrida en el flujo, con el nodo responsable y la acción tomada.	n8n: solo escritura (Create Page)

Ver también el Entregable 1 (Diagrama de arquitectura) para la vista visual del flujo completo.

2. Base: Centro de Comando

Es la base central del ecosistema: cada fila representa una **pieza de contenido** en algún punto de su ciclo de vida, desde la idea semilla hasta el contenido aprobado (o rechazado) listo para publicar. El *Notion Trigger* del flujo escucha nuevas filas agregadas aquí para arrancar la generación con IA.

Campo	Tipo (Notion)	Descripción	Valores / Ejemplo
id	Texto (auto)	Identificador único de la pieza de contenido.	3e664d4a-9063...
Idea_Semilla	Texto	Disparador manual: la idea o tema que el equipo quiere convertir en contenido.	"Post sobre freelancing en Miraflores"
Contenido_Generado	Texto largo	Resultado de Claude API (Message a Model) con el copy final generado.	Texto del post de LinkedIn

Campo	Tipo (Notion)	Descripción	Valores / Ejemplo
Estado	Select	Estado del ciclo de vida de la pieza.	Pendiente · Esperando aprobación · Aprobado · Rechazado
Errores	Relación / Texto	Referencia a incidentes ocurridos al procesar esta pieza (si los hubo).	Vacío o texto del error

Ciclo de vida típico de una fila:

- 1 El equipo crea una fila nueva con **Idea_Semilla** cargada (Estado = Pendiente).
- 2 El Notion Trigger de n8n detecta la fila y dispara el flujo de generación con RAG + Claude API.
- 3 n8n escribe el resultado en **Contenido_Generado** y cambia el Estado a "Esperando aprobación".
- 4 Se envía un correo de aprobación (HITL) por Gmail a Sam con el contenido generado.
- 5 Al aprobar o rechazar desde el correo, n8n actualiza el Estado final de la fila.

3. Base: Base RAG (Conocimiento)

Contiene los fragmentos de conocimiento de referencia (tono de marca, información de negocio, datos de contexto) que se usan como contexto (RAG) antes de llamar a Claude API. El nodo *Get many database pages* trae todas las filas (Return All) y el nodo *Code in JavaScript* las concatena en un único bloque de contexto (`rag_context`) que se inserta en el prompt.

Campo	Tipo (Notion)	Descripción	Valores / Ejemplo
Categoría	Select / Texto	Tema o tipo de conocimiento del fragmento.	"Tono de marca", "Datos de negocio"
Contenido	Texto largo	El fragmento de conocimiento en sí, tal como se inyecta en el prompt.	"Kolibri CRM atiende inmobiliarias boutique..."

Mantenimiento: agregar o editar filas acá cambia directamente la calidad y precisión del contenido generado, sin tocar el flujo de n8n. Es el mecanismo principal para "entrenar" el tono de la IA.

4. Base: Log de Errores

Registra cada falla real ocurrida durante la ejecución del flujo. Los tres nodos críticos del flujo (Claude API, Create/Update página en Centro de Comando, y Gmail HITL) tienen configurado *Retry on Fail* (3 intentos, 1000ms de espera) y, si aun así fallan, *Continue Using Error Output* — en lugar de cortar todo el flujo, cada uno escribe una fila acá antes de que la ejecución termine con seguridad.

Campo	Tipo (Notion)	Descripción	Valores / Ejemplo
Mensaje_Error	Texto largo	Mensaje de error crudo devuelto por el nodo que falló.	"The credential Gmail account needs to be reconnected"

Campo	Tipo (Notion)	Descripción	Valores / Ejemplo
Nodo_Fallido	Texto	Nombre del nodo de n8n donde ocurrió la falla.	"Send message and wait for response (notificación de aprobación por Gmail)"
Fecha	Fecha	Fecha (y hora) en que se registró el error.	2026-09-25
Accion_Tomada	Select	Qué hizo el sistema (o el equipo) frente al error.	Detenido y registrado (Break) · Reintentado (Resume) · Resuelto manualmente

5. Procedimientos operativos

5.1 Cargar una nueva pieza de contenido

- Abrir la base **Centro de Comando** en Notion.
- Crear una fila nueva y completar únicamente **Idea_Semilla**.
- Dejar **Estado** en su valor por defecto (Pendiente). El resto de los campos los completa el flujo automáticamente.

5.2 Revisar y aprobar contenido pendiente

- Revisar el correo de Gmail con el asunto "Kolibri CRM - Aprobación de contenido generado por IA".
- Leer el contenido generado incluido en el cuerpo del correo.
- Hacer click en "Approve" o "Disapprove" — esto reanuda el flujo pausado en n8n y actualiza el Estado en Notion automáticamente.

5.3 Actualizar el conocimiento de la IA (RAG)

- Abrir la base **Base RAG**.
- Agregar una fila nueva con Categoría + Contenido, o editar una existente.
- El cambio se refleja en la próxima ejecución del flujo (no requiere reiniciar nada en n8n).

5.4 Auditar e interpretar el Log de Errores

- Abrir la base **Log de Errores** y ordenar por Fecha descendente.
- Identificar el **Nodo_Fallido** para saber qué parte del sistema falló (IA, escritura en Notion, o notificación por Gmail).
- Leer **Mensaje_Error** para el detalle técnico (ej. credencial expirada, límite de la API, timeout).
- Si **Accion_Tomada** = "Detenido y registrado", la pieza de contenido quedó a medio procesar: revisar su fila en Centro de Comando y, de ser necesario, volver a disparar el flujo manualmente desde n8n.
- Errores recurrentes en el mismo nodo (ej. Gmail) suelen indicar credenciales OAuth vencidas — reconectar la credencial desde n8n (Credentials → Reconnect).

6. Buenas prácticas de mantenimiento

- No borrar filas del **Log de Errores**: sirve como historial de auditoría del sistema.
- No editar manualmente **Contenido_Generado** mientras el Estado sea "Esperando aprobación" — puede generar inconsistencia con el correo ya enviado.
- Revisar el Log de Errores al menos una vez por semana como parte del mantenimiento del ecosistema.

- Mantener la Base RAG curada y sin duplicados: contenido contradictorio ahí degrada la calidad de lo que genera la IA.